

Agentic AI 개념과 해커톤 과제 구현 방안

명지대학교 경영정보학과/AI-RPA 사업단
강영식 교수 (yskang@mju.ac.kr)



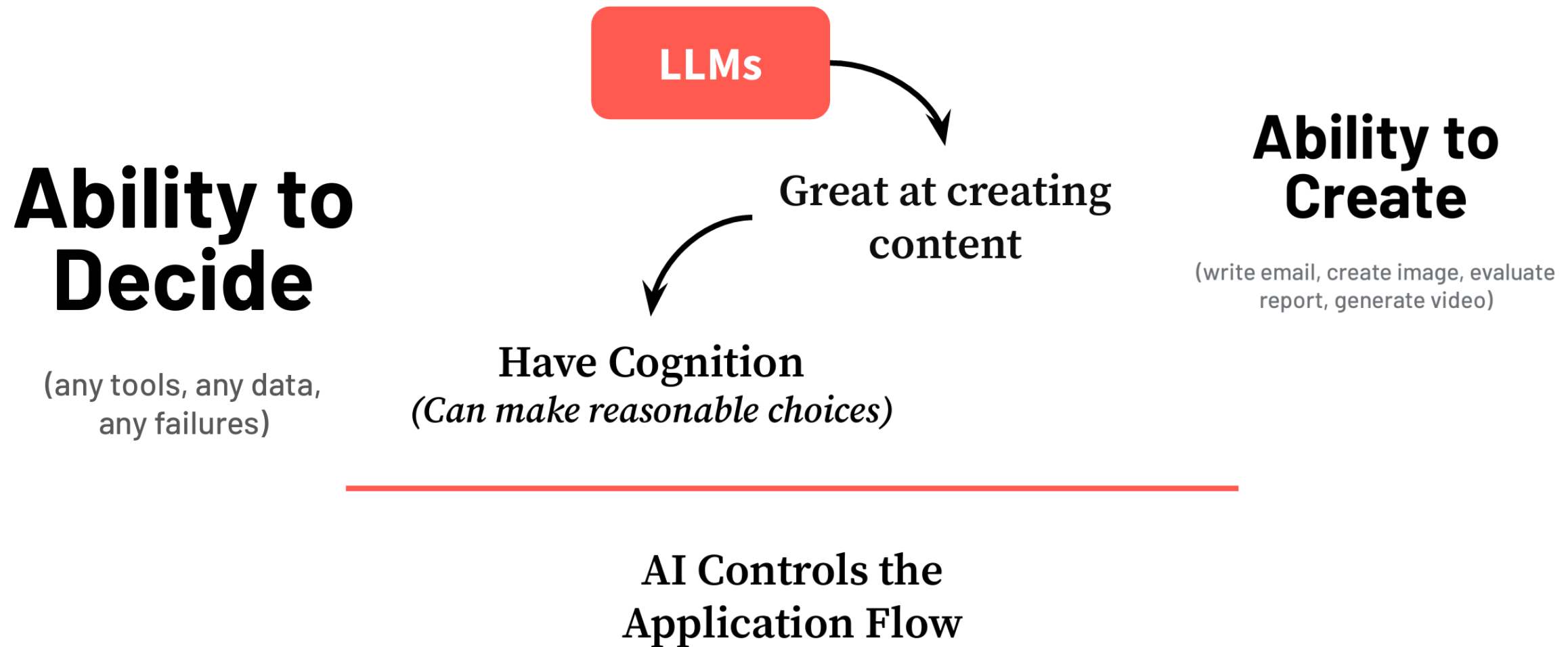
1. Agnetic AI와 Multi-agent System 개념

Agentic AI 개념과 폭발적 성장 이유

Agentic AI(에이전트가 스스로 계획하고 도구를 쓰고 여러 단계를 거쳐 목표를 달성하는 AI) 혁명이 2024 ~ 2025년에 폭발적으로 일어나는 이유는 5가지 핵심 조건이 동시에 충족되고 있기 때문임

번호	조건 (2024~2025년에 처음으로 동시에 충족)	구체적인 예시 & 왜 지금인가?
1	LLM이 충분히 똑똑해짐 (Reasoning 능력 폭발)	o1, Claude 3.5 Sonnet, GPT-4o, Grok-2 등 → Chain-of-Thought를 스스로 할 수 있을 정도로 똑똑해짐. 2022년 ChatGPT는 “생각”은 했지만 계획을 세우고 실행까지는 못 했음
2	도구 사용 (Tool Use / Function Calling)이 표준이 됨	OpenAI (2023.6), Anthropic, Google Gemini, Groq, Cohere 등 거의 모든 주요 LLM이 JSON function calling을 공식 지원 → AI가 외부 API, 브라우저, 코드 실행기를 자유롭게 호출 가능
3	메모리 + 장기 계획 기술이 실용화됨	ReAct, Reflexion, Plan-and-Execute, Tree-of-Thoughts, AutoGPT (2023.3) → BabyAGI → CrewAI, LangGraph 등 실제 프로덕션급 프레임워크가 2024년부터 폭발
4	비용이 극적으로 떨어짐	GPT-4o: \$2.5 / 1M tokens (GPT-4 Turbo보다 1/10 가격) + Llama 3.1 405B, DeepSeek-V3 등 오픈소스도 o1 수준 성능 → 하루에 수백 번 도구 호출하면서 에이전트를 돌려도 돈이 안 아픔
5	기업들이 자동화할 준비가 끝남	2020~2024년 동안 RPA + ChatGPT로 내부 프로세스를 문서화해 둔 기업들이 수십만 개 → 이제 “사람이 하던 3~4시간짜리 반복 업무”를 통째로 AI 에이전트에게 맡길 수 있는 인프라와 데이터가 쌓임

LLM의 역량 강화 → Agentic AI



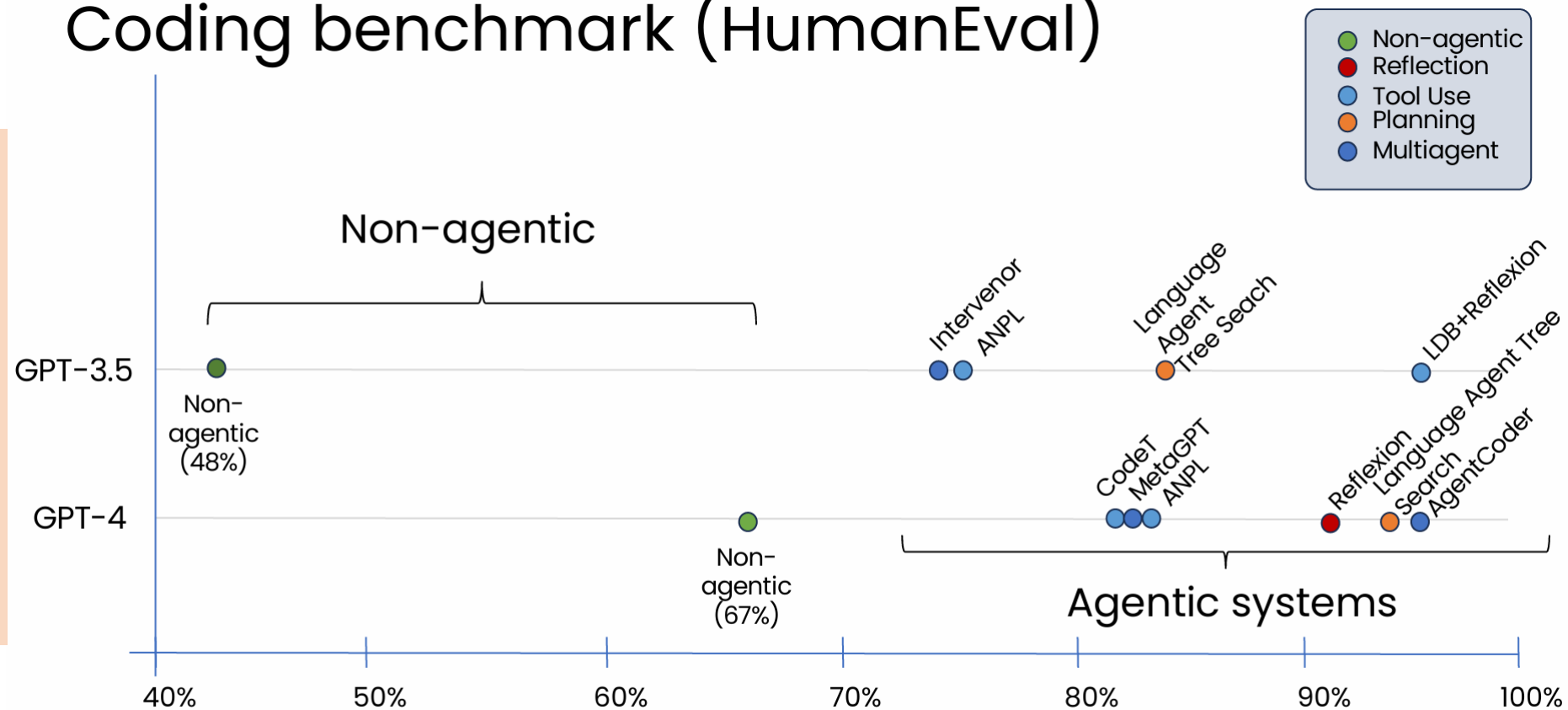
Spectrum of Agency



멀티 에이전트가 필요한 이유

- 역할 분리 → 성능과 품질 ↑
- 병렬처리 → 속도 ↑
- 모듈화: Tool 추가와 업데이트, 모델 교체
- 자기 확증 편향 감소
- 단일 실패 지점 제거 → 안정성 ↑
- 확장성 ↑

Coding benchmark (HumanEval)



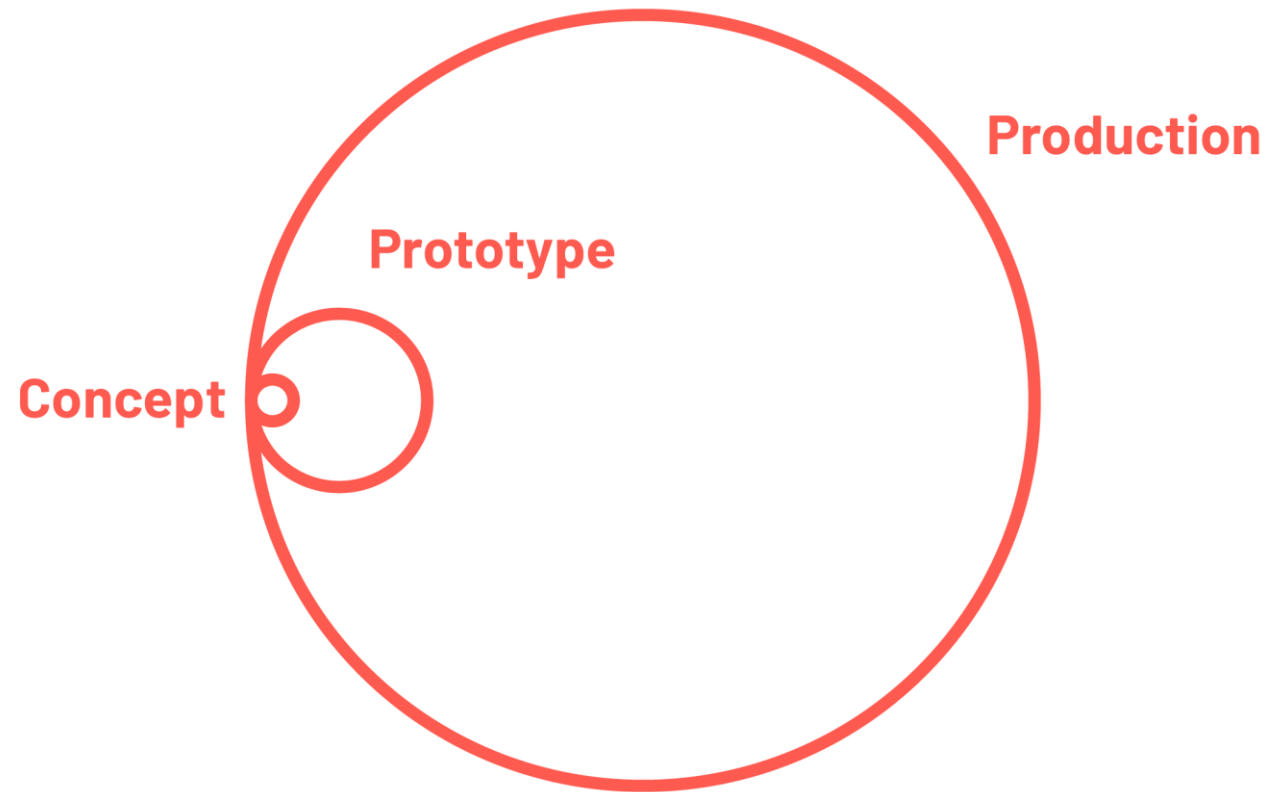
DeepLearning.AI

Andrew Ng

2. 해커톤 과제 구현 방안

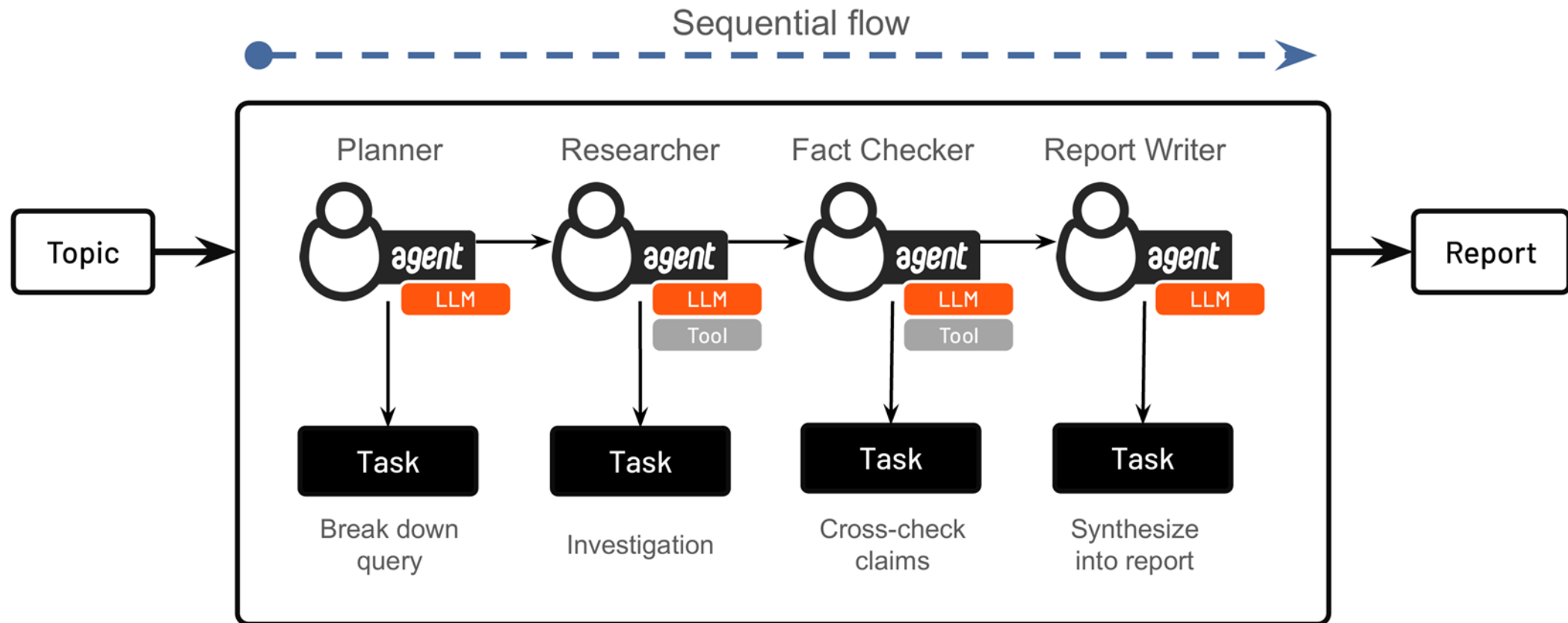
① 작동하는 핵심 기능을 빠르게 구현 후 개선해 나가는 방식 채택

해커톤에서는 과제 정의서(Concept)의 요구사항을 충족하는 프로토타입을 구축하는 것임. 일정 등을 고려해서 과제 정의서의 요구사항을 충족하는 핵심 기능을 빠르게 구현한 후에 개선해 나가는 방식이 적절함



② 멀티 에이전트 협업 방식 결정

멀티 에이전트 협업 방식은 Sequential, Hierarchical, Parallel, Hybrid가 있으나, 본 과제에서는 Sequential (또는 Hierarchical)을 적용하면 됨. 그런 다음에 필요한 에이전트 (Agent)와 태스크 (Task)를 결정함



Think like a **manager!**



The 80/20 Rule: Focus on Tasks Over Agents

When building effective AI systems, remember this crucial principle: 80% of your effort should go into designing tasks.

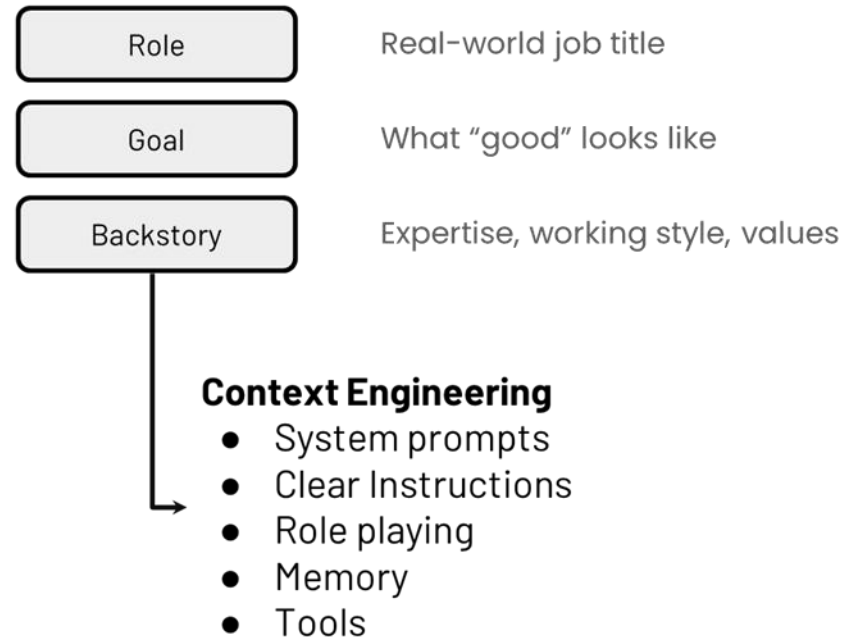
Even the most perfectly defined agent will fail with poorly designed tasks, but well-designed tasks can elevate even a simple agent.

- **80% effort:**
 - Craft clear tasks
- **20% effort:**
 - Polish agent personas

③ 프롬프팅을 통한 컨텍스트 엔지니어링 최적화 방안

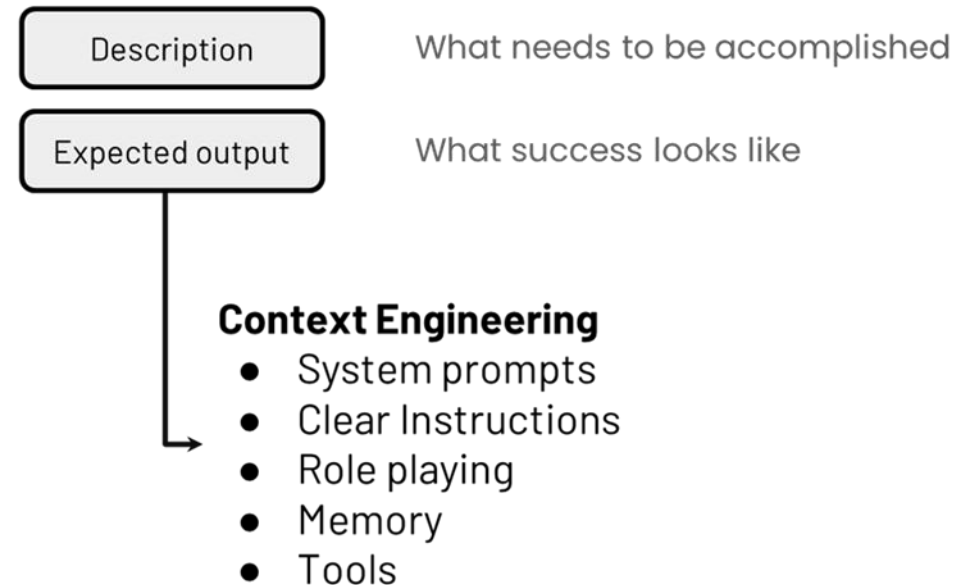
프롬프팅 (Prompting)을 통한 컨텍스트 엔지니어링 (Context Engineering)은 대규모 언어 모델 (LLM)이 더 정확하고 일관되며 유용한 답변을 생성하도록, 모델에게 제공되는 모든 배경 정보와 환경 (Context) 을 전략적으로 설계하고 최적화하는 기술

Agent



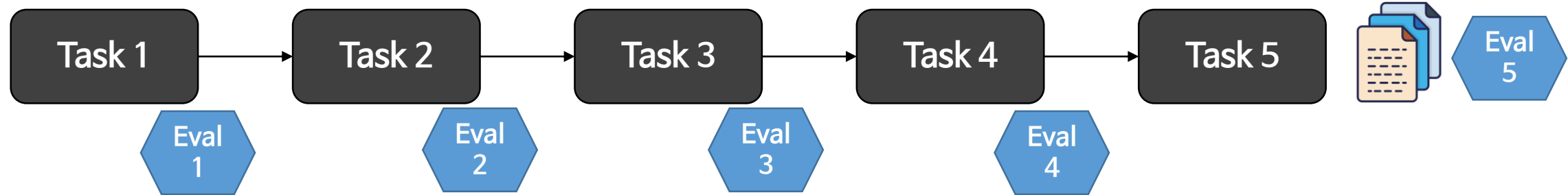
Task

- Single purpose, single output



④ 평가 방안 마련

해커톤 과제의 최종 산출물인 계약 공고문(템플릿)을 평가하고, 개별 태스크의 결과물을 평가하는 방안을 마련해야만 Agentic AI 시스템의 성능 개선을 파악할 수 있음. 평가 방안도 처음부터 잘 만들 필요가 없고, quick and dirty 방식으로 개발해야 함



‘Eval 5’를 통해서 최종 산출물을 평가해서 결과가 좋지 않으면 각 태스크의 산출물을 평가해서 성과가 좋지 않으면서 개선이 가능한 것을 먼저 찾아서 개선해 나감

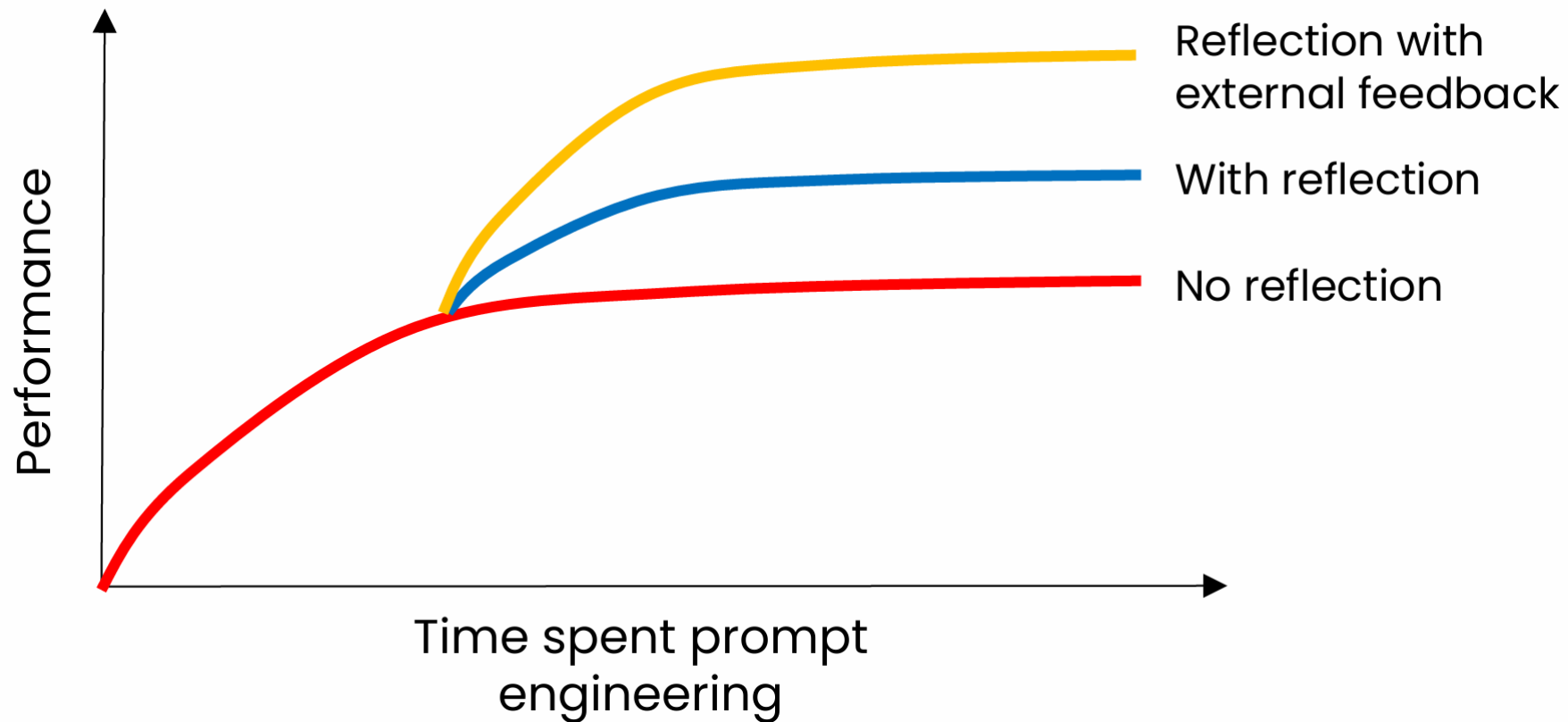
평가의 두 가지 축

	Evaluate with code (objective)	LLM-as-judge (subjective)
Per example ground truth (개별 사례마다 정답이 있음)	Checking invoice date extraction <pre>if (extracted_date == actual_date): num_correct += 1</pre>	Counting gold-standard talking points Count the number of gold standard points in the following text...
No per example ground truth (개별 사례마다 정답이 없음)	Checking marketing copy length <pre>if len(text) <= 10: num_correct += 1</pre>	Grading charts with a rubric Grade this chart according to (i) whether it has clear axes labels, (ii)

CrewAI의 경우, 일부 평가 방식을 가드레일 (Guardrails)을 통해 구현할 수 있음

⑤ 자기반성(reflection) 반영을 통한 성능 개선

Agentic Workflow에 자기반성(reflection)을 반영하면 성능 개선을 달성할 수 있음. CrewAI에서는 3가지 방식으로 자기반성(reflection)을 반영할 수 있음



⑤-1 Task 자체에 Reflection 포함하기

Agent를 하나만 쓰되, Task description에 “초안
→ 자기비평 → 수정본”을 강제함.
구현이 가장 간단하나, “자기평가가 봐주기식”이 될
수 있어 품질 상승 폭이 제한될 때가 있음

```
writer = Agent(  
    role="Writer",  
    goal="Produce high-quality Korean report",  
    verbose=True  
)
```

```
write_with_reflection = Task(  
    description="""  
Write the report in Markdown.
```

Reflection requirements:

- 1) First, create a draft.
- 2) Then critique your own draft using this checklist:
 - Missing sections?
 - Weak logic or unclear sentences?
 - Any unverifiable claims?
- 3) Finally, output ONLY the revised final version (no critique text).

Keep code blocks unchanged.

```
""",  
    agent=writer,  
    expected_output="Final revised Markdown report."  
)
```

⑤-2 멀티 에이전트를 통해 Reflection 구조화하기

Reflection을 Task로 분리하면 훨씬 안정적이거나,
Task가 늘어남(시간과 비용 증가)

```
writer = Agent(role="Writer", goal="Draft and revise", verbose=True)
critic = Agent(role="Critic", goal="Find issues and give rewrite instructions",
               verbose=True)
```

```
draft = Task(
    description="Write the first draft in Markdown.",
    agent=writer,
    expected_output="Markdown draft."
)
```

```
reflect = Task(
    description="""
Review the draft and output:
- Issues (bullets)
- Rewrite instructions (bullets)
- Score 0-10 and PASS/FAIL
""",
    agent=critic,
    expected_output="Critique + instructions + score."
)
```

```
revise = Task(
    description="Revise the draft using the critic's instructions. Output only
final Markdown.",
    agent=writer,
    expected_output="Final Markdown report."
)
```


⑤-3 Task Guardrail로 기준 미달 시 재시도

CrewAI 문서의 Guardrails(검증/재시도) 패턴을 이용하면, Reflection을 ‘합격 기준 게이트’로 만들 수 있음

- Writer가 결과를 냄
- guardrail()이 형식/누락/금지요소/길이 등을 체크
- FAIL이면 “수정해서 다시 제출”을 유도하며 재시도

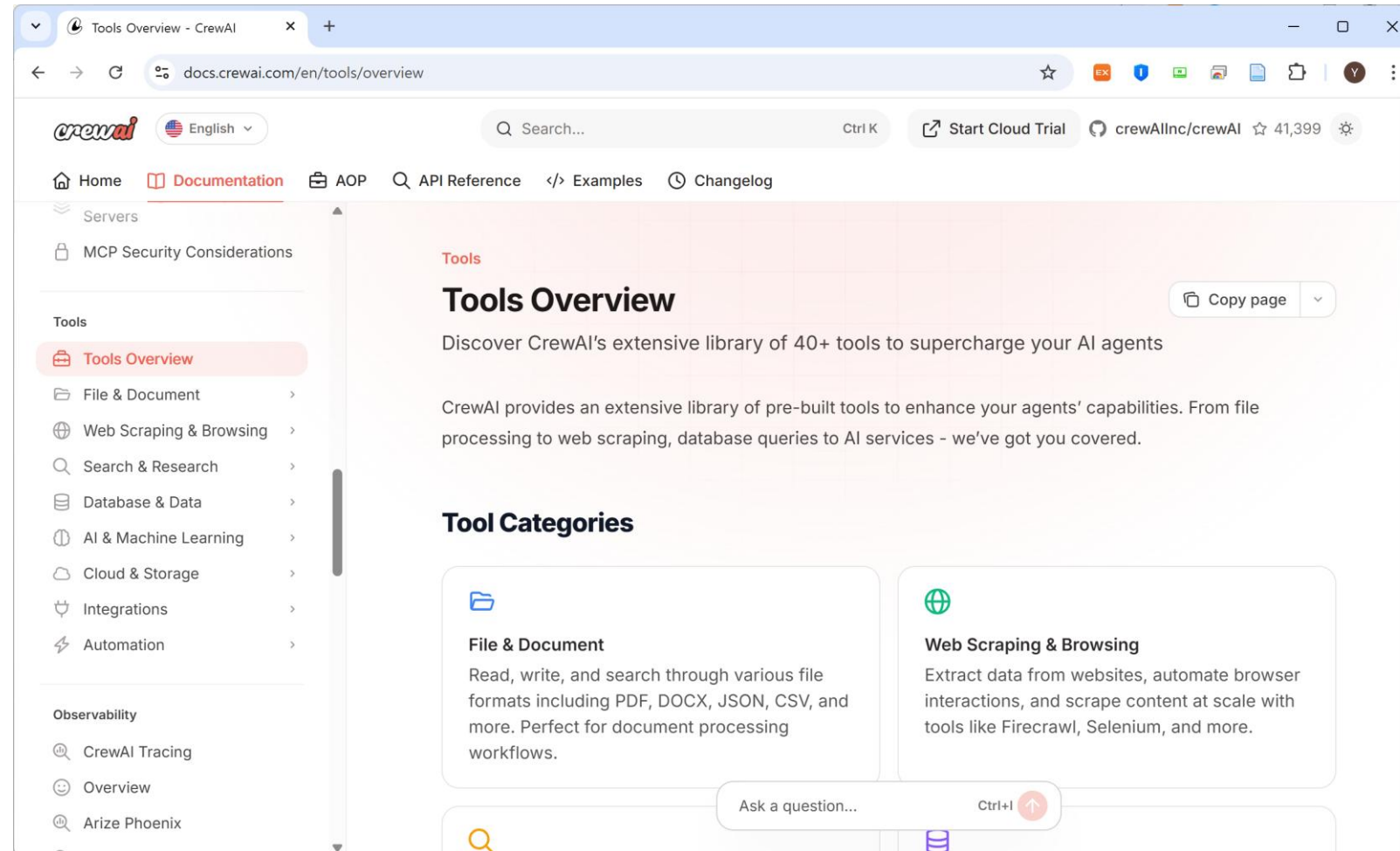
⑥ Tools 사용을 통한 성능 개선

에이전틱 시스템에서 Tools의 사용은 시스템의 능력, 효율성, 유연성을 극대화하는 데 있어 결정적인 역할을 함. CrewAI에서 3가지 방식으로 Tools를 사용할 수 있음

중요성	도구가 제공하는 이점	예시 도구
능력 확장	정적 지식의 한계를 넘어 실시간/전문 데이터 접근	검색 도구, 데이터베이스 쿼리 도구
정확성 향상	LLM의 계산 및 논리적 오류를 방지하고 정확성을 보장	계산기, Python 인터프리터
실행력	에이전트가 디지털 환경에서 실제 행동을 수행·자동화	이메일/캘린더 도구, API 호출 도구
협업 효율	에이전트 간 역할 분담을 명확히 하고 병렬 작업 지원	특화된 내부 함수/스크립트 도구

⑥-1 CrewAI에서 제공하는 Tools 사용

CrewAI는 File & Document, Web Scraping & Browsing, Search & Research 등에서 다양한 Tools를 제공하므로 이를 활용할 수 있음 (참조: <https://docs.crewai.com/en/tools/overview>)



⑥-2 사용자 정의 도구(Custom Tool)를 개발해서 사용함

사용자 정의 도구는 CrewAI 에이전트가 **고유한 내부 시스템, 데이터베이스, 또는 특정 비즈니스 로직**에 직접 접근하여 상호작용할 수 있게 함으로써, 에이전트의 능력을 단순한 추론에서 **실제 비즈니스 프로세스 자동화 및 실행**으로 확장시키는 데 필수적임. 이로써 에이전트는 일반적인 정보 검색을 넘어 **실질적인 업무 수행자** 역할을 할 수 있게 됨

BaseTool 방식

```
from crewai.tools import BaseTool

class MyCustomTool(BaseTool):
    name: str = "Name of my tool"
    description: str = "What this tool does. It's vital for effective utilization."
    args_schema: Type[BaseModel] = {}

    def _run(self, argument: str) -> str:
        # Your tool's logic here
        return "Tool's result"
```

@tool 방식

```
from crewai.tools import tool

@tool("sum_numbers")
def sum_numbers(a: int, b: int) -> str:
    """두 수를 더한다."""
    return str(a + b)
```

⑥-3 Code execution 기반 Tool 사용

CrewAI에서 code execution 기반 Tool은 “에이전트가 자연어로만 추론하지 않고, 필요할 때 파이썬 함수를 호출해 실제로 코드를 실행해서 결과를 얻는 방식”임.

즉, Agent(판단/지휘) ↔ Tool(실행/계산/입출력)로 역할이 분리됨

Tool 안에서 직접 무거운 작업을 돌리기보다, subprocess(별도 파이썬 프로세스) 나 Docker로 격리 실행하게 만들면 커널 크래시/무한루프 위험을 크게 줄일 수 있음

```
from crewai import Agent, Task, Crew, Process
from crewai.tools import tool
import pandas as pd
```

```
@tool("summarize_csv")
def summarize_csv(file_path: str) -> str:
    """
    CSV 파일 경로를 입력받아 기초 통계 요약(df.describe())을 문자열로 반환합니다.
    입력: file_path (예: "data/sales.csv")
    출력: 요약 통계 텍스트
    """
    df = pd.read_csv(file_path)
    return df.describe(include="all").to_string()
```

```
data_analyst = Agent(
    role="Data Analyst",
    goal="데이터를 실제로 계산/요약해서 근거 기반으로 답한다",
    backstory="필요하면 도구를 호출해 계산 결과를 확보한다.",
    tools=[summarize_csv],
    verbose=True,
)
```

```
analyze_task = Task(
    description=(
        "다음 CSV 파일을 분석해 핵심 요약 통계를 제시하십시오: data/sales.csv\n"
        "반드시 summarize_csv Tool을 사용해 계산 결과를 근거로 답하십시오."
    ),
    expected_output="기초 통계 요약과 간단한 해석",
    agent=data_analyst,
)
```

```
crew = Crew(
    agents=[data_analyst],
    tasks=[analyze_task],
    process=Process.sequential,
    verbose=True,
)
result = crew.kickoff()
print(result)
```

⑦ Memory를 통한 통제와 성능 강화

CrewAI의 Memory는 에이전트와 Crew가 작업하면서 생성한 맥락·결과·중간 산출물을 다음 추론에서 재사용할 수 있게 하는 저장하는 메커니즘을 의미함 (CrewAI가 메모리 저장소(기본: ChromaDB + SQLite 계열) 를 초기화/사용하면서, Windows에서 커널이 “조용히 죽는(ExitCode 3221225477 = 0xC0000005 접근 위반)” 케이스가 종종 발생함. ChatGPT 등을 활용해서 해결 필요)

Short Term

Stores data from past executions to add context that gets shared among agents

Long Term

Reflects on differences between expected outputs and actual outputs on tasks to improve agents through feedback

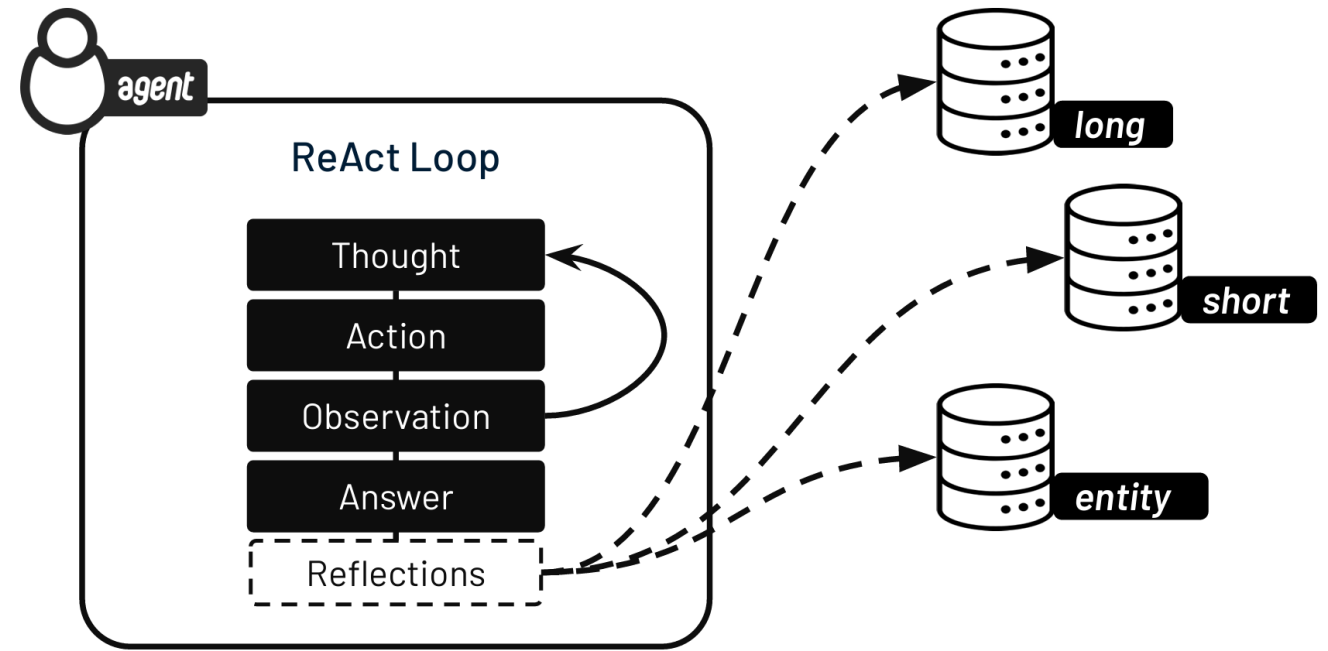
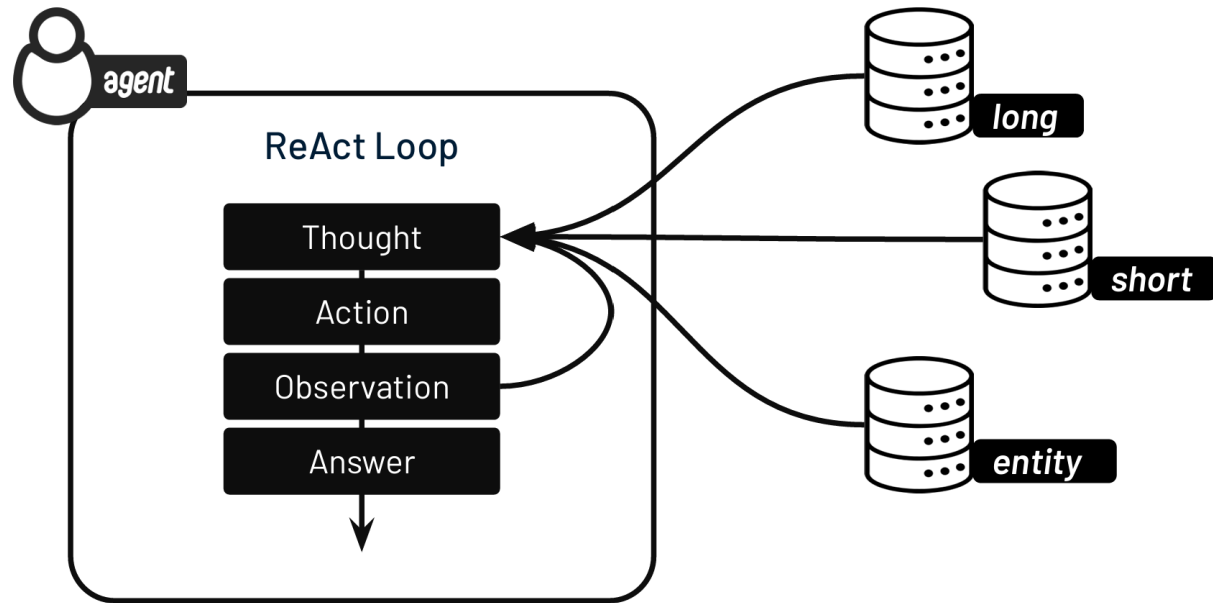
Entity

Collects facts about recognizable people, companies, locations, products, etc

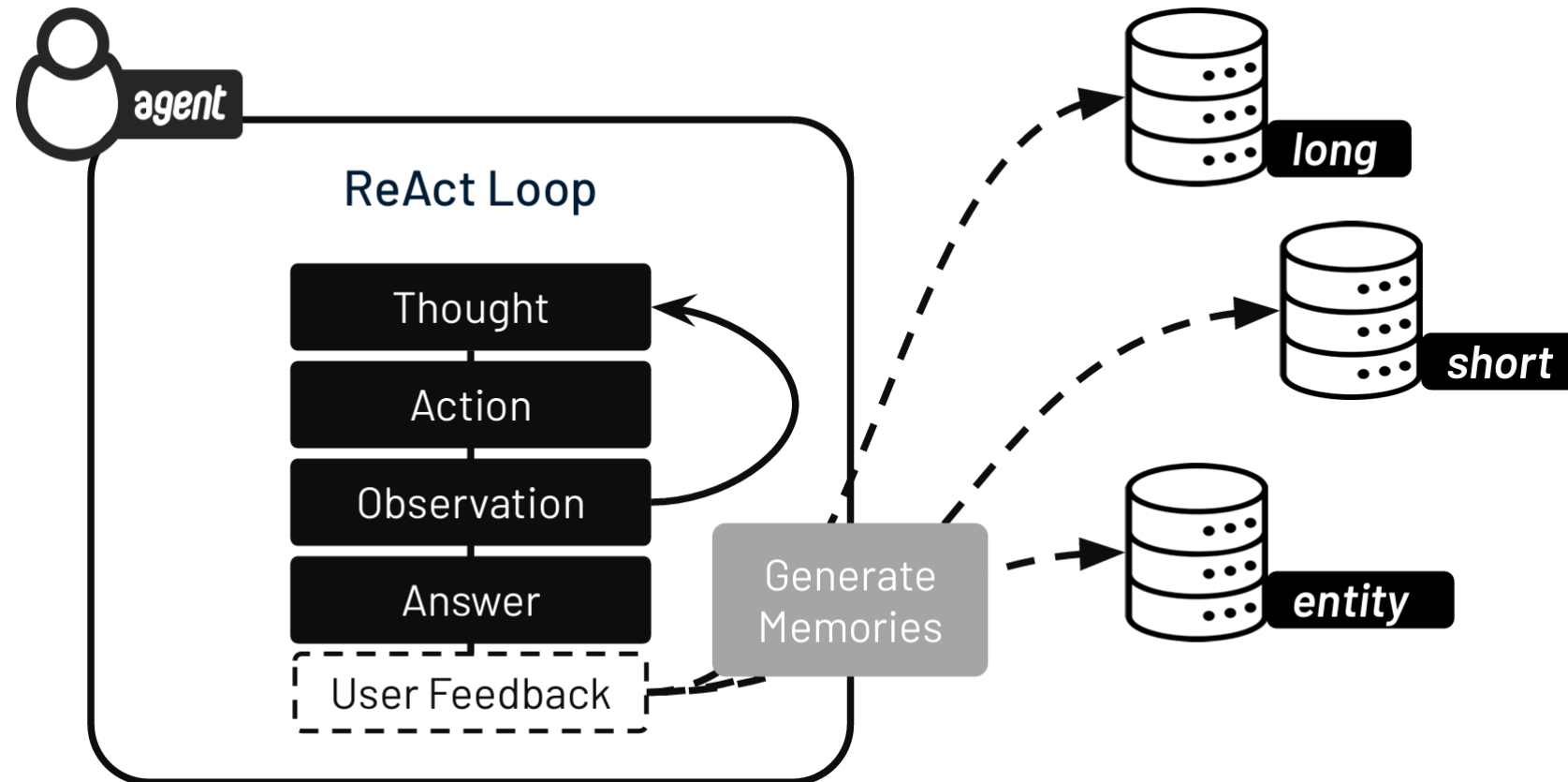
⑦ Memory를 통한 통제와 성능 강화

```
planning_crew = Crew(  
    agents=[code_explorer, documentation_planner],  
    tasks=[analyze_codebase, create_documentation_plan],  
    memory=True,  
    verbose=False  
)  
  
planning_crew.kickoff()
```

⑦ Memory를 통한 통제와 성능 강화



⑦ Memory를 통한 통제와 성능 강화



⑧ Knowledge를 통한 통제와 성능 강화

CrewAI의 Knowledge는 Agent들이 공통으로 참고하는 정적(또는 준정적) 지식 저장소(즉, 사전에 주어진 참고자료)를 말함.
(이에 반해, Memory는 '실행 중에 생기는 경험'을 의미) Knowledge는 회사 정책, 규정, 매뉴얼, 논문 등 “항상 사실로 간주해야 하는 정보”임 → Agent가 매번 같은 기준으로 판단하도록 만드는 장치

Memory

- Internal information adapted from previous executions
- Selectively added to agent's context during current execution
- Updated from feedback by human users or LLM-as-a-Judge

Knowledge

- External information retrieved from different sources
- Selectively added to agent's context from flat files or vector databases
- Not updated from feedback. Pre-filled at run-time.

Text Sources

- Raw Text
- Text Files
- PDF Documents

Structured Data

- CSV Files
- Excel Spreadsheets
- JSON Documents

⑧ Knowledge를 통한 통제와 성능 강화

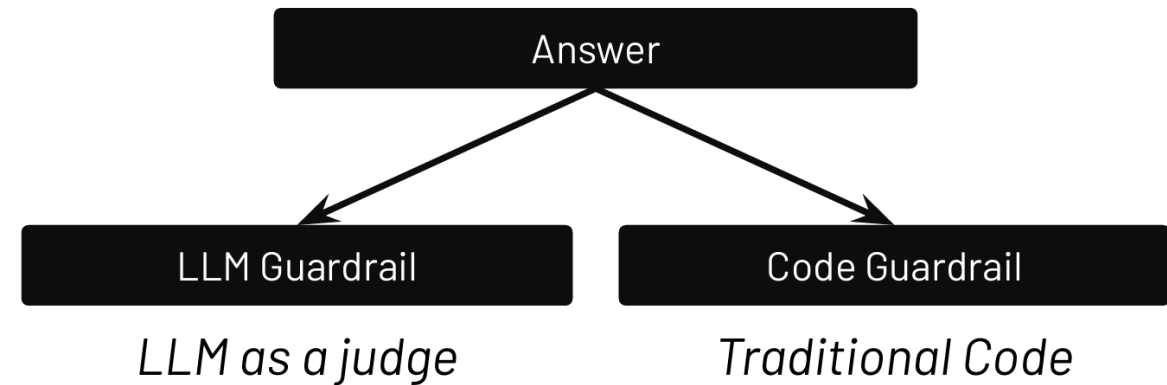
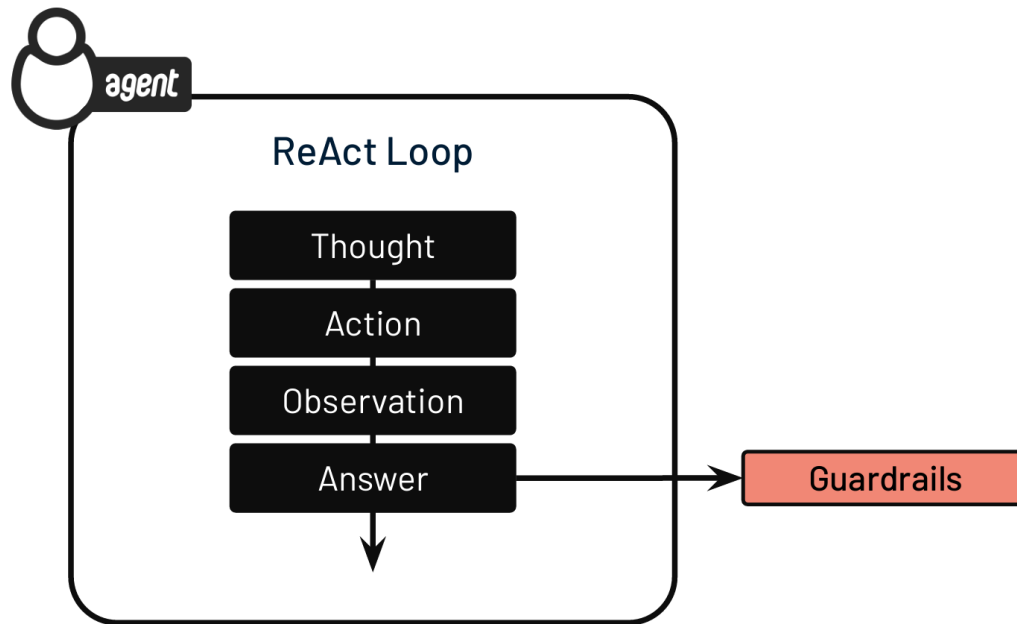
```
from crewai import Agent, Task
from crewai.knowledge.source.string_knowledge_source import StringKnowledgeSource

content = "User's name is John. He is 30 years old and lives in San Francisco."
string_source = StringKnowledgeSource(content=content)

agent = Agent(
    role="About User",
    goal="You know everything about the user.",
    backstory="You are a master at understanding people and their preferences.",
    knowledge_sources=[string_source],)
```

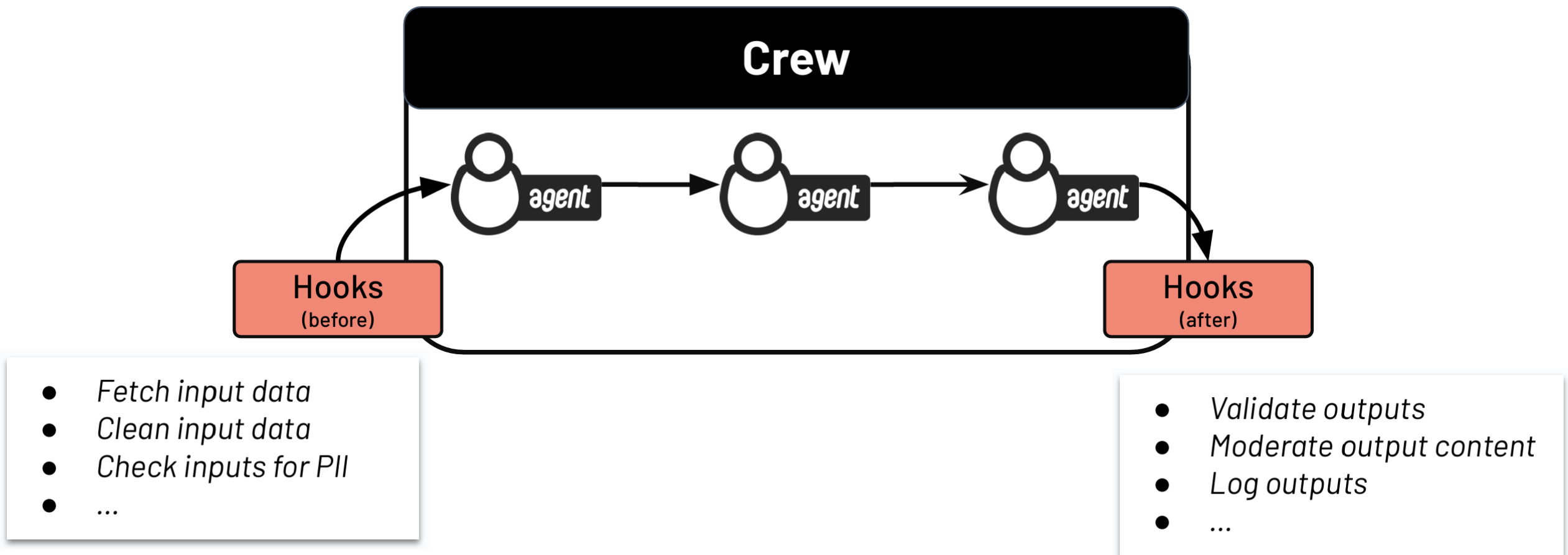
⑨ Guardrails를 통한 통제와 성능 강화

Guardrails는 “Agent가 자유롭게 추론하되, ‘넘지 말아야 할 경계’를 강제하는 장치”를 의미함. Agentic AI가 비결정적 (nondeterministic)이기 때문에 신뢰성과 반복 가능성을 확보하기 위해 꼭 필요한 개념임



⑩ Execution Hooks를 통한 통제와 성능 강화

Execution Hooks는 “Crew 실행의 전후에 자동으로 실행되는 부가 로직”을 말함. Agentic 시스템에서는 Agent 로직과 Tool 로직에 넣기 애매한 공통 작업이 많음. 이러한 공통 작업이 Execution Hooks를 통해 실행됨



감사합니다

